SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Problem Statement:	**Bus Ticket Reservation System (BusBuddy: A Smart Bus Ticket Reservation System)**		
Student Name:	K. Sampath Reddy	**Reg. No.:**	192424285
Date:	15-10-2026	**Duration:**	60 minutes

Code of Conduct

I K Sampath Reddy (192424285) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K Sampath Reddy

BusBuddy: Architecture Design Report

A hybrid layered-modular and event-driven architecture for a Bus Ticket Reservation System

Architecture at a glance

Aspect	Decision
Architectural style	**Hybrid:** client-server, layered modular monolith, with event-driven background workers
Seat consistency	**Double-Gate Seat Guard:** Redis lock (8 min, auto-expiry) + PostgreSQL UNIQUE rule on trip and seat
Data	PostgreSQL (source of truth), Redis (locks, sessions, cache), object storage (PDF tickets, exports)
Integration	Adapters for payment, maps and messaging; signed webhooks; message queue with transactional outbox
Deployment	Stateless application instances behind a load balancer, spread over two availability zones
Evolution	Clear module boundaries and events, so heavy modules can be extracted into services when needed

Report roadmap

The report follows the six questions of Annexure A in order. The table shows where each question is answered and which rubric criterion it supports.

Sec.	What the section covers	Rubric criterion (weight)
1	System objectives, architecturally significant functional and non-functional requirements, capacity estimate, quality attributes, constraints	System Requirement Analysis (30%)
2	Candidate architectural styles, weighted evaluation, the selected hybrid style and why it fits	Selection of Software Architecture (25%)
3	Overall software architecture diagram with layers, databases, external systems, APIs and communication	Architecture Diagram and Component Design (20%)
4	Component responsibilities, communication mechanisms, events and the end-to-end workflow	Component Interaction and Quality Attribute Analysis (15%)
5	How the design achieves scalability, security, performance, reliability, maintainability and availability	Component Interaction and Quality Attribute Analysis (15%)
6	Architecture decisions, trade-offs, future enhancements, integration and long-term maintainability	Architectural Justification and Technical Presentation (10%)

List of figures

Figure	Title	Section
Figure 1	Quality attribute utility tree for BusBuddy	1.5
Figure 2	Weighted evaluation of candidate architectural styles	2.2
Figure 3	Overall software architecture of BusBuddy	3.1
Figure 4	Booking workflow as a sequence diagram	4.3
Figure 5	Life cycle of a seat (state diagram)	4.3
Figure 6	Deployment view for scalability and availability	5.7

1. Analyze System Requirements

1.1 Problem statement and system objectives

In the scenario from Assessment Tool 1, Meera books a seat by phone and finds at the counter that the same seat was sold to someone else. Payment and refunds are manual, the manager cannot see real occupancy, and the conductor has no reliable passenger list. The problem is therefore **not** to show a list of buses. It is to give every user the same correct, real-time view of seats, to sell each seat only once, and to handle payment, tickets and refunds without manual work.

An architecture for this system must make one thing impossible (selling a seat twice) and make everything else fast, observable and easy to change. The six objectives below come from the SRS, and the last column shows what each one asks of the architecture.

ID	Objective	Measure of success	What it asks of the architecture
O-1	Fast, simple booking	Median search-to-ticket time of 3 minutes or less	Light clients, cached search, few round trips per booking
O-2	Zero double booking	No seat sold twice	One shared seat inventory, seat lock with expiry, database uniqueness rule
O-3	Secure payment and automatic refund	98% payment success; instant refund amount	Payment adapter, verified webhooks, refund worker with retries
O-4	Paperless, verifiable tickets	100% of tickets have a unique QR code	Ticketing module with signed QR; scanner that works offline
O-5	Operator control and insight	Live sales and occupancy dashboard	Reporting module reading from a read replica; role-based access
O-6	Safe, reliable, scalable service	99.5% uptime; 5,000 concurrent users	Stateless instances, load balancing, redundancy in two zones

1.2 Functional requirements that shape the architecture

BusBuddy has 47 functional requirements (FR-01 to FR-47). Not all of them affect the structure of the system equally. The groups below are the **architecturally significant** ones: they decide which components exist and how they talk to each other.

Requirement group	FR IDs	What the system must do	Architectural impact
Live seat map and seat lock	FR-12 to FR-16	Show one real-time view of seats; hold chosen seats for 8 minutes; never confirm a seat twice	Dedicated Seat Inventory module; Redis locks with expiry; database UNIQUE rule; WebSocket push
Payment and confirmation	FR-22 to FR-26	Confirm only after gateway success; allow retry; send ticket within 1 minute	Payment adapter; signed, idempotent webhook handler; ticket delivery done asynchronously
Cancellation and refund	FR-28 to FR-31, FR-40	Calculate refund by slab; refund to original method; resell the seat immediately	Refund worker with retries; slabs held in configuration; SeatReleased event
Search and discovery	FR-07 to FR-11	Fast filtered search by route and date	Search module with caching; read replica for heavy reads
Notifications and tracking	FR-26, FR-32 to FR-35	Send SMS and email, delay alerts and live location	Event bus and notification worker; maps adapter
Admin and reports	FR-36 to FR-42	Manage buses, routes, trips, fares; dashboards and exports	Fleet module; Reporting module on a read replica; role-based access control
Conductor verification	FR-43, FR-44	Scan QR codes, even with poor network; update trip status	Ticket verification API; signed QR; offline scan queue in the scanner app
Security and audit	FR-45 to FR-47	Audit log, hashed passwords, session timeout	Identity module; append-only audit table; sessions held in Redis

1.3 Non-functional requirements that shape the architecture

The 22 non-functional requirements (NFR-01 to NFR-22) have measurable targets, which makes them useful as design drivers. The table groups the ones that influence the structure most.

NFR	Requirement and target	Effect on the architecture
NFR-01, 03	Search in 3 s (95th percentile, 1,000 users); booking confirmed within 5 s of payment	Cache popular searches; keep the booking path short; move slow work (SMS, PDF) out of the request
NFR-02	Seat map refreshes within 2 s of a change	Push updates through a WebSocket channel instead of waiting for a page reload
NFR-04, 05, 07	TLS 1.2+; no card data stored; OWASP Top 10 protection	Payment page hosted by the gateway; WAF at the edge; validated input; layered security
NFR-06	OTP valid 5 minutes, 5 requests per hour	Rate limiting at the gateway; OTP state in Redis with expiry
NFR-08	Zero double booking through a database uniqueness rule	One transactional database as the source of truth; UNIQUE rule is the final gate
NFR-09	Payment without booking reconciled within 15 minutes	Webhook plus a reconciliation worker; idempotent processing
NFR-10, 11	RPO 1 h, RTO 4 h; 99.5% monthly uptime	Instances in two zones; database standby; automated backups
NFR-12	SMS failure falls back to email within 2 minutes	Notification worker with retries and channel fallback
NFR-16, 17	5,000 concurrent users; 500 buses; 100,000 bookings a month	Stateless instances behind a load balancer; shared Redis; room to add servers
NFR-18, 19, 20	80% unit-test coverage; rules changed by configuration; alert within 5 minutes	Modular code with clear boundaries; configuration store; central logging and alerting
NFR-14, 21	Mobile-first from 360 px; latest browsers, Android 9+, iOS 14+	Responsive progressive web apps instead of separate native apps
NFR-22	Consent and deletion within 30 days (DPDP Act, 2023)	Personal data kept in the Identity module so it can be anonymised cleanly

1.4 Load and capacity estimate

A rough estimate shows what kind of workload the architecture must carry. The figures use the SRS targets and the assumptions stated in the table.

Quantity	Estimate	How it was worked out
Bookings per day	about 3,300	100,000 bookings per month ÷ 30 days (NFR-17)
Peak booking rate	about 0.1 per second; up to about 2 per second on a festival day	60% of daily bookings fall in a 6-hour evening window (about 2,000 ÷ 21,600 s); a festival day is assumed to be 20 times busier
Seat-lock attempts at peak	about 10 per second	Assuming 1 in 5 lock attempts ends in a paid booking
User requests at peak	about 1,000 per second	5,000 concurrent users (NFR-16) × one request every 5 seconds, mostly search and seat-map reads
Share of read traffic	over 99%	Search and seat-map views greatly outnumber bookings
Data stored in 3 years	about 11 GB	100,000 bookings × about 3 KB (booking, seats, payment, audit) × 36 months (DR-07)

> **What this tells us**
> The workload is **read-heavy** and the write volume is **small**. One PostgreSQL primary with a cache and a read replica handles this comfortably, so raw throughput is not the hard part. The hard part is **correctness under contention**: many people wanting the same seat on the same trip at the same moment. This insight drives the choice of architecture in Section 2.
>
> These are planning assumptions. As suggested in Assessment Tool 1, they should be replaced with real operator data before launch.

1.5 Key quality attributes

The quality attributes were ranked by how much damage a failure would do to BusBuddy. Reliability comes first because a double-booked seat is the exact problem the system exists to remove. Figure 1 turns each attribute into measurable scenarios (a "utility tree") and rates every scenario for importance and difficulty.

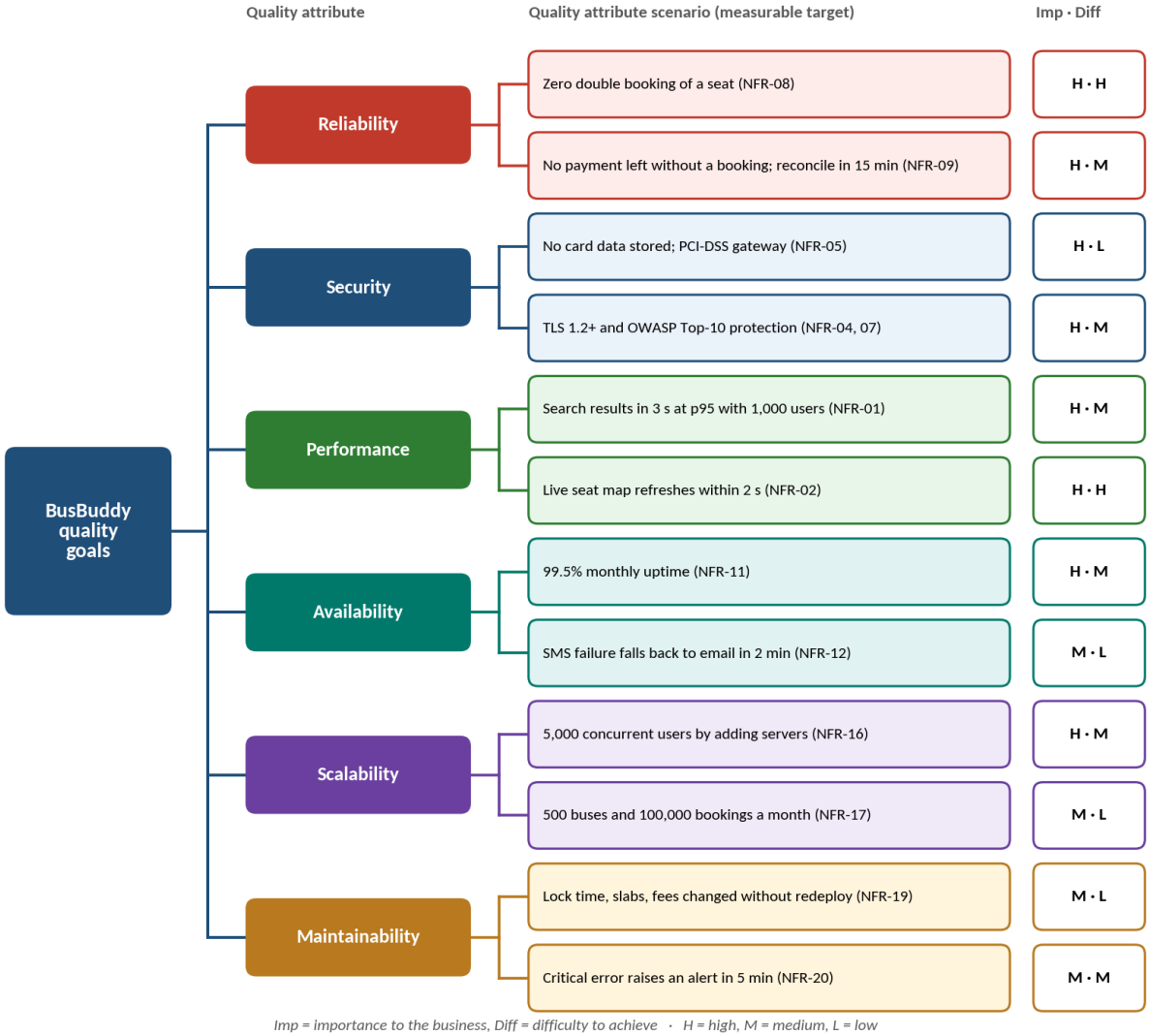


Figure 1: *Quality attribute utility tree: six attributes, twelve measurable scenarios*

Rank	Quality attribute	Why it matters for BusBuddy	Main target
1	Reliability	A double-booked seat or a payment without a ticket destroys trust	0 duplicate seat sales; reconcile in 15 min
2	Security	The system handles payments and personal data	PCI-DSS gateway; TLS 1.2+; DPDP Act
3	Performance	The seat map and search must feel instant to win the booking	3 s search (p95); 2 s seat-map refresh

Rank	Quality attribute	Why it matters for BusBuddy	Main target
4	Availability	Tickets can only be sold while the system is up; evenings are busy	99.5% monthly uptime
5	Scalability	Festival and weekend peaks are many times the normal load	5,000 users by adding servers
6	Maintainability	Rules such as fees and refund slabs change; the project has 12 weeks	80% test coverage; rules by configuration

1.6 Constraints that influence the design

Constraint (from Assessment Tool 1)	Consequence for the architecture
T-1: an external payment gateway must be used, within its API limits	Payment is isolated behind an adapter so the gateway can be replaced and its limits respected
T-2: standard cloud hosting and an open-source database	PostgreSQL, Redis and RabbitMQ are chosen; no proprietary database features
T-3: pages of 1 MB or less for slow mobile networks	Lightweight progressive web apps, a CDN and caching
T-4: seat locking needs an in-memory cache with expiry	Redis is a mandatory component and is made redundant
E-1, E-2: limited budget; per-transaction provider fees	One deployable application, open-source tools, few moving parts
TM-1, TM-2: six sprints (12 weeks); Must-haves by Sprint 3	A modular monolith is far quicker to build and test than a set of distributed services
L-1, L-2: DPDP Act and PCI-DSS	Card data never touches BusBuddy; personal data is kept in one module

2. Select a Suitable Software Architecture

2.1 Candidate architectural styles

Six styles from the assignment list were considered. Client-server and MVC are not listed separately because they describe the split between clients and server and the structure of the user interface. Every candidate uses them in some form.

Style	What it offers	Why it is not enough for BusBuddy on its own
Layered monolith	Simple, quick to build; one database transaction keeps data consistent	Scales only as a whole; a slow SMS or refund call inside a request can slow bookings; weak fault isolation
Microservices	Each service scales and deploys independently	A booking would span seat, payment and booking services, so consistency needs distributed transactions (sagas); heavy operations effort for a 12-week project
SOA with ESB	Reuse of shared enterprise services	Heavy middleware and governance that a single-operator system does not need
Serverless	Pay per use and automatic scaling	Cold starts, hard to hold seat locks and WebSocket connections, vendor lock-in, harder to debug a multi-step booking
Pure event-driven	Loose coupling and resilience	Eventual consistency is not acceptable for seat allocation, and the flow is harder to follow and test
Hybrid: layered modular monolith + event-driven workers	Strong consistency for the core, loose coupling for the edges	**Selected.** Keeps the strengths of the layered style and adds events only where they help

2.2 Weighted evaluation

Each style was scored from 1 (poor) to 5 (excellent) against seven criteria taken from the requirements. The weights reflect the quality ranking in Section 1.5: consistency and delivery speed count most. The scores are the author's engineering judgement based on the Assessment Tool 1 requirements.

Criterion	Weight	Layered monolith	Micro-services	SOA + ESB	Server-less	Event-driven	Hybrid (chosen)
Strong consistency for seat booking	25%	5	2	3	2	2	5
Delivery speed and simplicity (12 weeks)	20%	5	1	2	3	2	4
Scalability	15%	3	5	4	5	4	4
Maintainability and evolution	15%	3	5	4	3	4	4
Availability and fault isolation	10%	2	5	3	4	4	4
Cost and operations effort	10%	5	1	2	4	3	4
Integration with external services	5%	3	4	5	4	5	5
Weighted score (out of 5)	**100%**	**4.00**	**3.00**	**3.10**	**3.30**	**3.05**	**4.30**

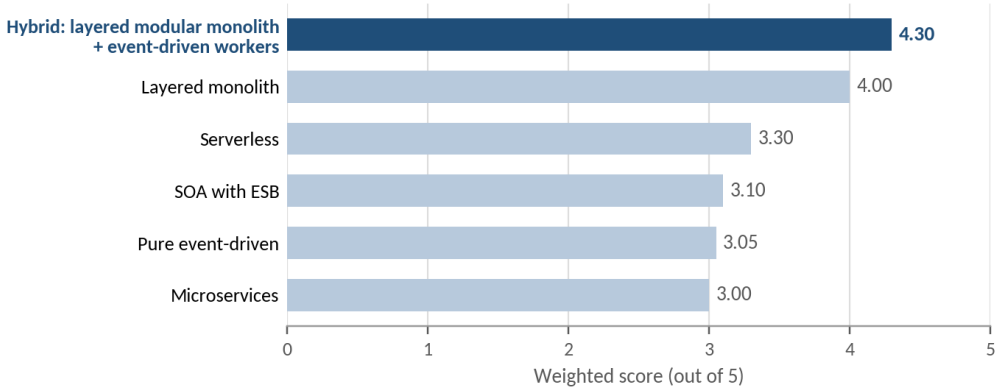


Figure 2: *Weighted scores of the candidate styles (higher is better)*

The hybrid scores highest (4.30). The plain layered monolith is second (4.00): excellent for consistency, simplicity and cost, but weaker on fault isolation and on handling external services. Event-driven workers fix exactly those two weaknesses at a small cost in complexity. Microservices score lowest because their strengths (independent scaling and deployment) are not needed at this load, while their weaknesses (distributed consistency, operations effort) hit the most important requirements.

2.3 The selected architecture

BusBuddy uses a **Hybrid architecture: a layered modular monolith with event-driven integration**. It combines five well-known ideas, each used where it helps most.

Idea	Where it is used in BusBuddy	What it gives
Client-server	Three clients (passenger, admin, conductor) talk to one backend over HTTPS REST and WebSocket	One source of truth; thin clients; easy to add new clients
Layered architecture	API layer, business layer, integration and data access layer, data tier; calls only go downward	Separation of concerns; each layer can be tested and changed on its own
Modular monolith	Ten business modules (seat, booking, payment and so on) in one deployable application with strict boundaries	One ACID transaction for a booking; simple deployment; easy to split later
Event-driven integration	Message queue and workers for notifications, refunds, reconciliation, waitlist and exports	Slow or failing external services never block a booking; automatic retries
Stateless, shared-nothing instances	Identical application instances; state kept in PostgreSQL and Redis	Add servers to scale; lose one and nothing is lost

2.4 Why this architecture is the most suitable

1. **The hardest requirement is consistency.** Confirming a booking and assigning its seats must happen in one transaction (FR-16, NFR-08). Inside a monolith this is a normal database transaction. Across microservices it would need distributed sagas, which are slower to build and easy to get wrong.
2. **The load is read-heavy and modest.** Section 1.4 shows about 2 bookings per second at a festival peak. Stateless instances, a cache and a read replica are enough, so the main benefit of microservices (scaling parts independently) is not needed yet.
3. **External systems are the main risk.** Payment, SMS and maps services can be slow or down. Handling them through events with retries means an SMS outage never blocks a booking (NFR-09, NFR-12).
4. **Time and budget favour simplicity.** Six sprints, one team and a limited budget (TM-1, E-1) point to one deployable application and open-source components.
5. **Growth stays possible.** Module boundaries and events let heavy modules be moved into their own services later, without changing the rest (Section 6.3).

3. Draw the Software Architecture Diagram

3.1 Overall architecture

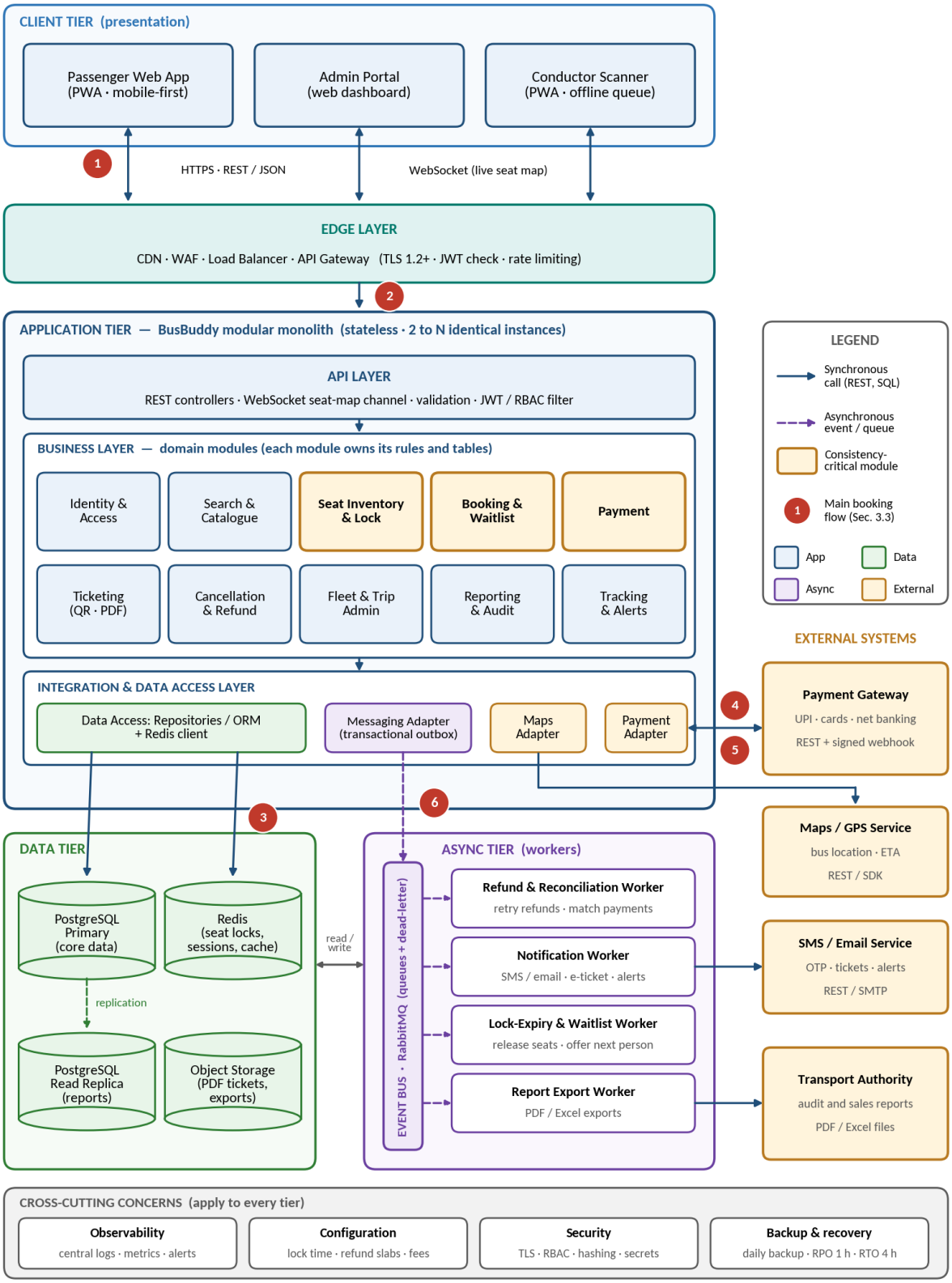


Figure 3: *Overall software architecture of BusBuddy (tiers, components, data stores, external systems and communication)*

3.2 How to read the diagram

Element	Meaning
Tiers (boxes with a coloured border)	Blue: client and application. Teal: edge. Green: data. Purple: asynchronous processing. Gold: external systems. Grey: cross-cutting concerns
Solid arrow	Synchronous call: the caller waits for the answer (REST over HTTPS, SQL, key-value commands)
Dashed purple arrow	Asynchronous event: the sender does not wait; a worker handles the message later
Gold module boxes	Consistency-critical modules (Seat Inventory, Booking, Payment) that carry the no-double-booking rule
Red numbered circles 1 to 6	The main booking flow, explained in Section 3.3

3.3 The main booking flow on the diagram

Step	What happens	Path on the diagram
1	A client calls the system over HTTPS (REST and JSON); the live seat map uses a WebSocket	Client tier to Edge layer
2	The edge layer ends TLS, applies WAF and rate limits, checks the JWT and forwards the request to a healthy instance; the API layer checks the role and input	Edge layer to API layer
3	The Seat Inventory module locks the seats in Redis for 8 minutes; core data is read and written through the Data Access layer	Business layer to Redis and PostgreSQL
4	The Payment module asks the gateway, through the Payment Adapter, to create a payment; the customer pays on the gateway's own page	Payment Adapter to Payment Gateway
5	The gateway sends a signed webhook; the Payment module verifies it and the Booking module commits the booking, with the database UNIQUE rule as the final check	Gateway to Payment Adapter to PostgreSQL
6	The booking writes an event to the outbox in the same transaction; the event reaches the event bus and workers send the e-ticket by SMS and email	Outbox to Event bus to Workers to SMS / Email

Layer rules that keep the structure clean:

- A lower layer never calls an upper layer; calls only go downward.
- Modules talk to each other only through their public service interface, never by reading another module's tables.
- Only the Data Access layer talks to PostgreSQL and Redis; only the adapters talk to external vendors.
- Anything slow or unreliable (SMS, email, refunds, exports) runs in a worker through an event, not inside the user's request.

4. Explain Components and Their Interactions

4.1 Component responsibilities

The first table lists the components that serve user requests. Technologies are examples that fit the constraints in Section 1.6; the architecture does not depend on a particular vendor.

Component	Responsibility	Requirements	Example technology
Client tier			
Passenger Web App	Search, live seat map with countdown, traveller details, payment, e-ticket download, booking history, bus tracking; mobile-first and accessible	FR-07 to FR-27, NFR-14, 15	React PWA
Admin Portal	Manage buses, routes, trips, fares and refund slabs; dashboards and report export	FR-36 to FR-42	React web app
Conductor Scanner	Scan QR tickets with the phone camera; update trip status; queue scans offline and sync later	FR-43, FR-44	PWA on Android
Edge layer			
CDN, WAF, Load Balancer	Serve static files, block common attacks, end TLS, check instance health and spread traffic	NFR-04, 07, 11	Cloud load balancer
API Gateway	Route requests, validate JWT, rate-limit per user and address, log requests	NFR-06, 07	Nginx or cloud gateway
Application tier: API and business layers			
API Layer	REST controllers, WebSocket seat-map channel, input validation, role checks	NFR-07	Spring Boot
Identity & Access	Registration, OTP verification, login, lockout, sessions, roles, password hashing	FR-01 to FR-06, 36, 46, 47	Spring Security
Search & Catalogue	Trip search, filters, sorting, nearby dates; caches popular searches	FR-07 to FR-11	Service + Redis cache
Seat Inventory & Lock	Live seat map; 8-minute locks; 6-seat limit; women-safe seat rule; broadcasts seat changes	FR-12 to FR-17	Service + Redis
Booking & Waitlist	Traveller details, boarding point, fare summary, booking state, waitlist queue	FR-16, 18 to 21	Service + PostgreSQL
Payment	Create payment orders, verify signed webhooks, idempotency, trigger reconciliation	FR-22 to FR-24, NFR-09	Service
Ticketing	Booking ID, signed QR code, PDF e-ticket, QR verification	FR-25, 27, 43	Service
Cancellation & Refund	Cancel booking, calculate refund from slabs, request refund, release seats	FR-28 to FR-31, 40	Service
Fleet & Trip Admin	Buses with seat layouts, routes, boarding points, trips, fares	FR-37 to FR-39	Service
Reporting & Audit	Dashboards, exports, append-only audit log	FR-41, 42, 45	Service + read replica
Tracking & Alerts	Bus location and ETA, delay detection, trip status from the conductor	FR-32 to FR-34, 44	Service
Application tier: integration and data access layer			
Payment and Maps Adapters	Hide vendor APIs behind our own interfaces; add timeouts, retries and circuit breakers	T-1, NFR-09	REST clients
Messaging Adapter (outbox)	Writes events to an outbox table in the same transaction as the business change, then publishes them reliably	NFR-09, 12	Outbox pattern
Data Access	Repositories and ORM; transactions; Redis client; the only code that touches the data stores	NFR-08, 18	JPA / Hibernate

The second table covers background workers, data stores and external systems.

Component	Responsibility	Requirements	Example technology
Asynchronous tier			
Event Bus	Holds events in queues with acknowledgement, retry and a dead-letter queue	NFR-09, 12	RabbitMQ
Notification Worker	Sends OTPs, e-tickets, reminders and delay alerts; falls back from SMS to email	FR-26, 33 to 35, NFR-12	Worker process
Refund & Reconciliation Worker	Retries refunds; matches payments with bookings; refunds unmatched payments	FR-30, NFR-09	Worker process
Lock-Expiry & Waitlist Worker	Releases expired or abandoned seats; offers released seats to the waitlist for 15 minutes	FR-15, 20	Worker process
Report Export Worker	Builds PDF and Excel reports without slowing the dashboard	FR-42	Worker process
Data tier			
PostgreSQL Primary	System of record: users, trips, bookings, seats, payments, audit log; UNIQUE rule on trip and seat	NFR-08, DR-01	PostgreSQL
PostgreSQL Read Replica	Serves reports and heavy searches; standby for failover	NFR-11	PostgreSQL
Redis	Seat locks with expiry, sessions, OTP state, search cache, seat-change notifications	FR-14, DR-02, T-4	Redis
Object Storage	PDF tickets, exported reports, backups	FR-27, 42, NFR-10	S3-compatible storage
External systems			
Payment Gateway	Processes UPI, card and net-banking payments and refunds; PCI-DSS compliant; sends signed webhooks	FR-22 to FR-24, 30	e.g. Razorpay
SMS / Email Service	Delivers OTPs, tickets and alerts	FR-26, NFR-12	e.g. MSG91, SendGrid
Maps / GPS Service	Provides bus location and estimated arrival time	FR-32	e.g. Google Maps
Transport Authority	Receives audit and sales reports as exported files	FR-42, 45	File export

4.2 How components communicate and exchange data

From to	Mechanism	Data exchanged	Why this choice
Clients to Edge / API	HTTPS, REST and JSON; WebSocket for the seat map	Requests, responses, seat-status updates	Simple, cacheable and secure; push gives the 2-second refresh
API Layer to modules	In-process method calls (synchronous)	Data transfer objects	Fast; allows one transaction across modules
Modules to PostgreSQL	SQL through the ORM over TLS	Bookings, seats, payments, audit rows	ACID transactions and UNIQUE rules
Seat module to Redis	Key-value commands (SET with NX and EX)	Lock key trip + seat, holder, 480 s expiry	Atomic; expires on its own
Payment Adapter and Gateway	HTTPS REST to create orders; signed webhook for the result	Order ID, amount, status, signature	No card data touches BusBuddy; result is verifiable

From to	Mechanism	Data exchanged	Why this choice
Modules to Event Bus	Events written to an outbox, then published	BookingConfirmed, SeatReleased, RefundRequested, TripDelayed	No lost events; booking does not wait for slow work
Event Bus to Workers	Queue consumption with acknowledgement and retry	Event payloads	At-least-once delivery; handlers are idempotent
Notification Worker to SMS / Email	HTTPS REST or SMTP	Message text, ticket link	Falls back to email if SMS fails (NFR-12)
Tracking module to Maps	HTTPS REST or SDK	Coordinates, ETA	Vendor-neutral through the Maps Adapter
Conductor app to API	HTTPS REST with an offline queue	Ticket ID, scan time, trip status	Works on poor networks; syncs later
Redis to clients	Redis Pub/Sub through the WebSocket channel	Seat ID and new status	Every open seat map updates within 2 s

Core events. The event bus carries a small, stable set of events. They are the contract between modules and workers.

Event	Published by	Consumed by	Result
BookingConfirmed	Booking	Notification Worker	E-ticket sent by SMS and email within 1 minute (FR-26)
SeatReleased	Seat Inventory, Cancellation	Waitlist Worker, seat-map channel	Waitlist offer made; all open maps updated
RefundRequested	Cancellation & Refund	Refund Worker	Refund sent to the gateway; status tracked; retries on failure
PaymentUnmatched	Payment	Reconciliation Worker	Payment checked against bookings; refunded within 15 minutes if no booking
TripDelayed	Tracking & Alerts	Notification Worker	Delay alert to all booked passengers (FR-33)
ReportRequested	Reporting & Audit	Report Export Worker	PDF or Excel file created and stored

4.3 Overall system workflow

Figure 4 follows one successful booking from seat selection to e-ticket. It is split into three phases. The numbers in the figure match the steps in the table below it.

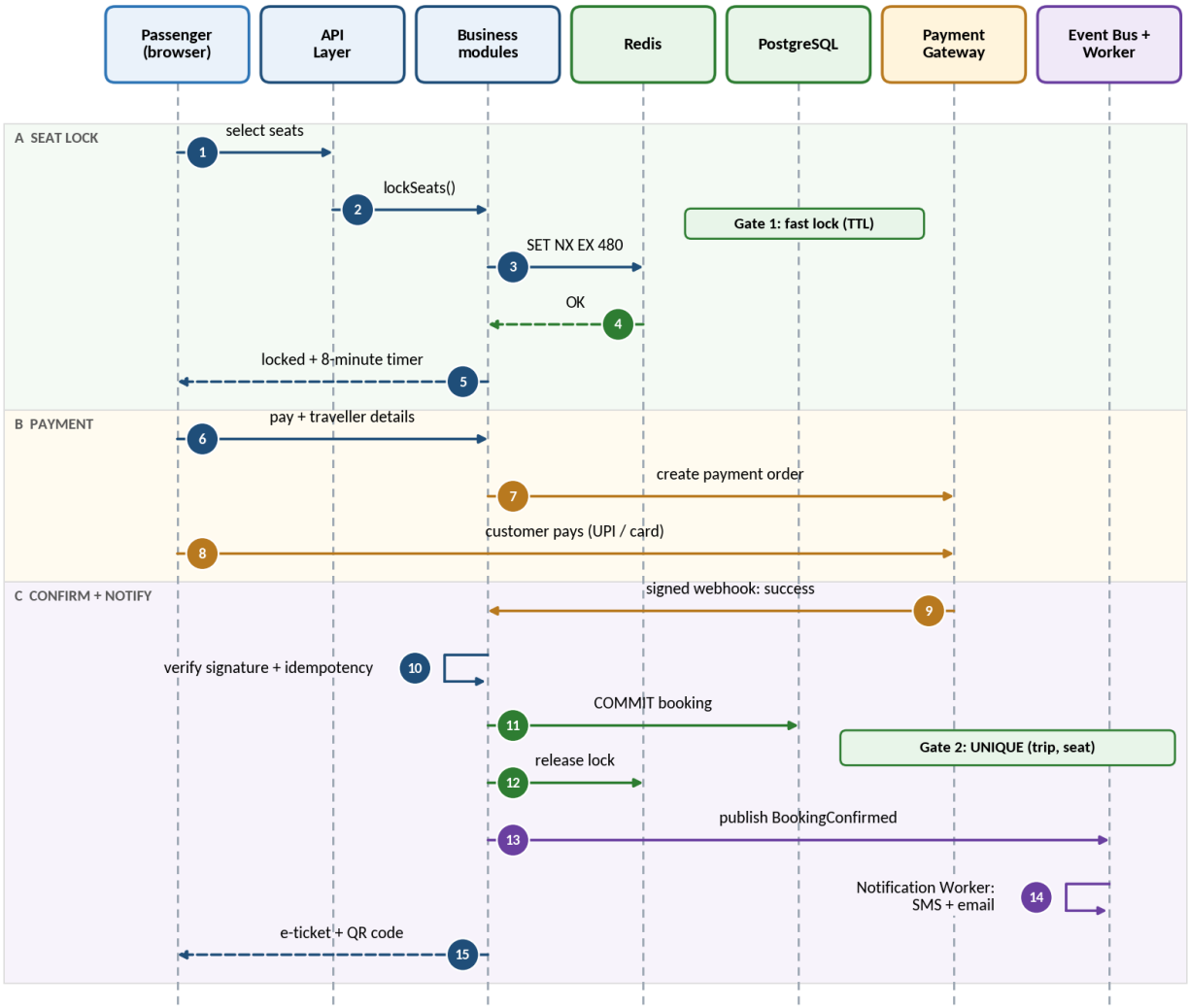


Figure 4: *Booking workflow: seat lock, payment and confirmation with notification*

Phase	Steps	What happens
A Seat lock	1 to 5	The passenger selects up to 6 seats. The Seat module asks Redis for an atomic lock that expires after 8 minutes (Gate 1). Redis answers OK, and the passenger sees the seats locked with a countdown timer.
B Payment	6 to 8	The passenger submits traveller details and pays. The Payment module creates an order at the gateway and the customer pays on the gateway's page, so BusBuddy never sees card data.
C Confirm and notify	9 to 15	The gateway sends a signed webhook. BusBuddy verifies the signature and checks the transaction ID has not been seen before (idempotency), then commits the booking. The database UNIQUE rule on trip and seat is Gate 2. The lock is released, a BookingConfirmed event is published, the Notification Worker sends the ticket, and the e-ticket with its QR code appears for the passenger.

> **Why two gates?**
> Gate 1 (Redis) is fast and absorbs the crowd: most competing users are turned away in a few milliseconds without touching the database. Gate 2 (PostgreSQL UNIQUE) is slower but cannot be bypassed: even if Redis lost a lock or a bug let two requests through, the database would refuse the second booking. Together they give speed **and** safety.

Figure 5 shows the life cycle of a single seat. Every transition is driven by the events described above, and every path respects the rule that one seat of one trip belongs to only one Locked or Booked booking.

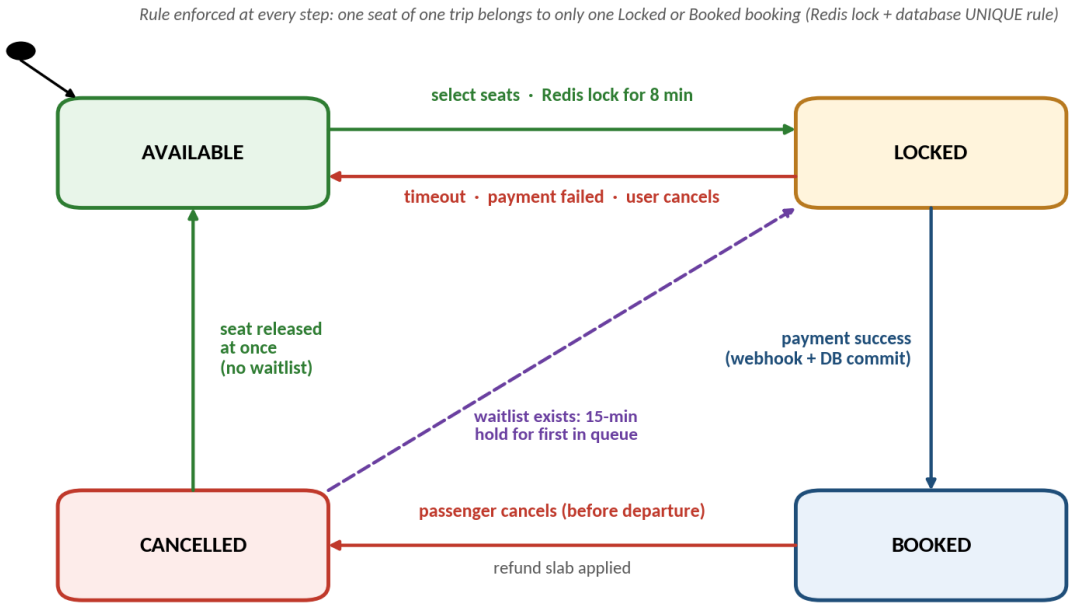


Figure 5: *Life cycle of a seat: available, locked, booked and cancelled*

Other workflows and failure paths. A good architecture is judged by how it behaves when things go wrong. The table shows how BusBuddy handles the main failure and alternate paths from the SRS use cases.

Scenario	How the architecture handles it
Payment fails or the user abandons	The lock stays until its 8-minute timer ends, so the user can retry (FR-24). When the Redis key expires the Lock-Expiry Worker publishes SeatReleased and every open seat map updates.
Payment succeeds after the lock expired	The Booking module cannot safely confirm the seat, so the payment is marked unmatched and the Reconciliation Worker refunds it automatically (UC-06, NFR-09).
Webhook arrives twice or not at all	The gateway transaction ID acts as an idempotency key, so a second delivery is ignored. If no webhook arrives, the Reconciliation Worker asks the gateway for the payment status.
Cancellation and refund	One transaction marks the booking Cancelled and frees the seat. RefundRequested goes to the Refund Worker, which applies the configured slab, calls the gateway and retries on failure; the admin is alerted after 3 failures (UC-10).
Conductor has no network	The signed QR can be checked on the phone against the downloaded manifest. Scans are stored locally and synchronised when the network returns (UC-22).
SMS provider is down	The Notification Worker retries with back-off, then sends the message by email within 2 minutes (NFR-12). Messages that still fail go to the dead-letter queue and raise an alert.
Trip is delayed	The conductor sets the status, or the Tracking module detects a delay of more than 15 minutes. A TripDelayed event makes the Notification Worker alert all booked passengers (FR-33).

5. Discuss Quality Attributes

This section checks the architecture against the six attributes named in the assignment. For each one, the table lists the design tactics, how they work in BusBuddy, and which requirement they satisfy. The attributes follow the priority order from Section 1.5, with scalability and availability supported by the deployment view in Figure 6.

5.1 Scalability

Verdict: the system scales horizontally by adding identical, stateless instances, and the read-heavy load is absorbed by caching and a read replica.

Design tactic	How it works in BusBuddy	Requirement
Stateless application instances	Sessions and locks live in Redis, not in instance memory, so any instance can serve any request and more can be added without code changes	NFR-16
Load balancing and auto-scaling	The load balancer spreads traffic; an auto-scaling group (minimum 2, maximum 6 instances) adds instances when CPU passes 60%	NFR-16
Caching and read replica	Popular searches are cached in Redis for 30 to 60 seconds; reports and heavy reads go to the replica, leaving the primary free for bookings	NFR-01, 17
Independent worker scaling	Workers read from queues and can be scaled on queue length, separately from the web instances	NFR-16
Headroom for growth	About 11 GB over 3 years fits one database comfortably; indexes on trip and date keep queries fast as data grows	NFR-17

5.2 Security

Verdict: security is applied in layers (defence in depth) so that no single failure exposes payments or personal data.

Design tactic	How it works in BusBuddy	Requirement
Encrypted transport	All traffic uses HTTPS with TLS 1.2 or higher; internal links to the database, Redis and queue are also encrypted	NFR-04
No card data	Payment happens on the gateway's PCI-DSS page; BusBuddy stores only the transaction ID and a masked reference	NFR-05, DR-05
Authentication and roles	JWT tokens, password hashing with salt, OTP checks, account lockout after 5 failed logins, role-based access for passenger, admin and conductor, session timeouts	FR-03 to FR-05, 36, 46, 47
Attack protection	WAF at the edge, rate limiting per user and address, parameterised queries and output encoding against SQL injection and cross-site scripting	NFR-06, 07
Verifiable messages	Payment webhooks are verified by signature; QR codes carry a signed token so forged tickets are rejected	FR-43, NFR-07
Audit and privacy	Append-only audit log for admin and payment actions; consent and deletion handled in the Identity module; secrets kept in a secrets store	FR-45, NFR-22, DR-09

5.3 Performance

Verdict: the booking path is kept short, reads are served from memory, and slow work is moved out of the user's request.

Design tactic	How it works in BusBuddy	Requirement
Caching	Search results and trip details are cached in Redis; static files are served from the CDN	NFR-01
Push instead of poll	Seat changes are pushed over WebSocket through Redis Pub/Sub, giving a 2-second refresh without heavy polling	NFR-02
Short booking path	After payment success only the database commit and outbox write happen in the request; SMS, email and PDF run in workers	NFR-03
Efficient data access	Indexes on route, date and trip; connection pooling; pagination of result lists	NFR-01
Light clients	Progressive web apps with pages of 1 MB or less, lazy-loaded images and compressed responses	T-3, NFR-14

5.4 Reliability

Verdict: correctness is protected by two independent gates, safe retries and automatic reconciliation.

Design tactic	How it works in BusBuddy	Requirement
Double-Gate Seat Guard	Redis lock (Gate 1) plus PostgreSQL UNIQUE rule on trip and seat (Gate 2) make a duplicate seat sale impossible	FR-16, NFR-08
ACID booking transaction	Booking, seat assignment, payment record and outbox event are committed together or not at all	NFR-08, 09
Idempotent handlers	Webhooks and event handlers can be repeated safely because every operation carries an idempotency key	NFR-09
Transactional outbox and retries	Events are never lost between the database and the queue; failed messages are retried and finally parked in a dead-letter queue	NFR-09, 12
Reconciliation worker	Payments without bookings are found and refunded within 15 minutes	NFR-09
Backup and recovery	Daily backups plus continuous log shipping give a recovery point of 1 hour; a rebuilt system is expected within 4 hours	NFR-10

5.5 Maintainability

Verdict: clear modules, strict layer rules and configuration-driven behaviour keep changes small and safe.

Design tactic	How it works in BusBuddy	Requirement
Modular structure	Each module has one responsibility and a public interface; layer rules stop modules reaching into each other's data	NFR-18
Configuration, not code	Lock time, refund slabs, convenience fee, session timeouts and fare limits are read from a configuration store	NFR-19
Automated testing and delivery	Unit tests on business logic (target 80%), integration tests with test containers, and a build pipeline that deploys with a rolling update	NFR-18
Observability	Central logging, metrics and alerting raise an alert within 5 minutes of a critical failure	NFR-20
Documented decisions	Architecture decisions are recorded (Section 6.1) so new team members understand why the design looks as it does	NFR-18

5.6 Availability

Verdict: there is no single point of failure in the core path, and failures of external services degrade gracefully. A 99.5% target allows about 3.6 hours of downtime in a 30-day month (0.5% of 720 hours).

Design tactic	How it works in BusBuddy	Requirement
Redundancy across two zones	At least two application instances in different availability zones behind the load balancer; Figure 6 shows the layout	NFR-11
Database and Redis failover	PostgreSQL standby with synchronous replication and automatic promotion; Redis replica with automatic failover	NFR-10, 11
Graceful degradation	If Redis is unavailable the Seat module falls back to a database lock row with an expiry time: slower but still safe	FR-14, NFR-08
Failure isolation for vendors	Timeouts, retries and circuit breakers in the adapters; SMS failure falls back to email	NFR-12
Planned maintenance	Rolling deployments with no downtime; heavy changes only in the 2 AM to 4 AM window	NFR-11, O-2
Monitoring	Health checks and uptime monitoring remove unhealthy instances automatically and notify the team	NFR-20

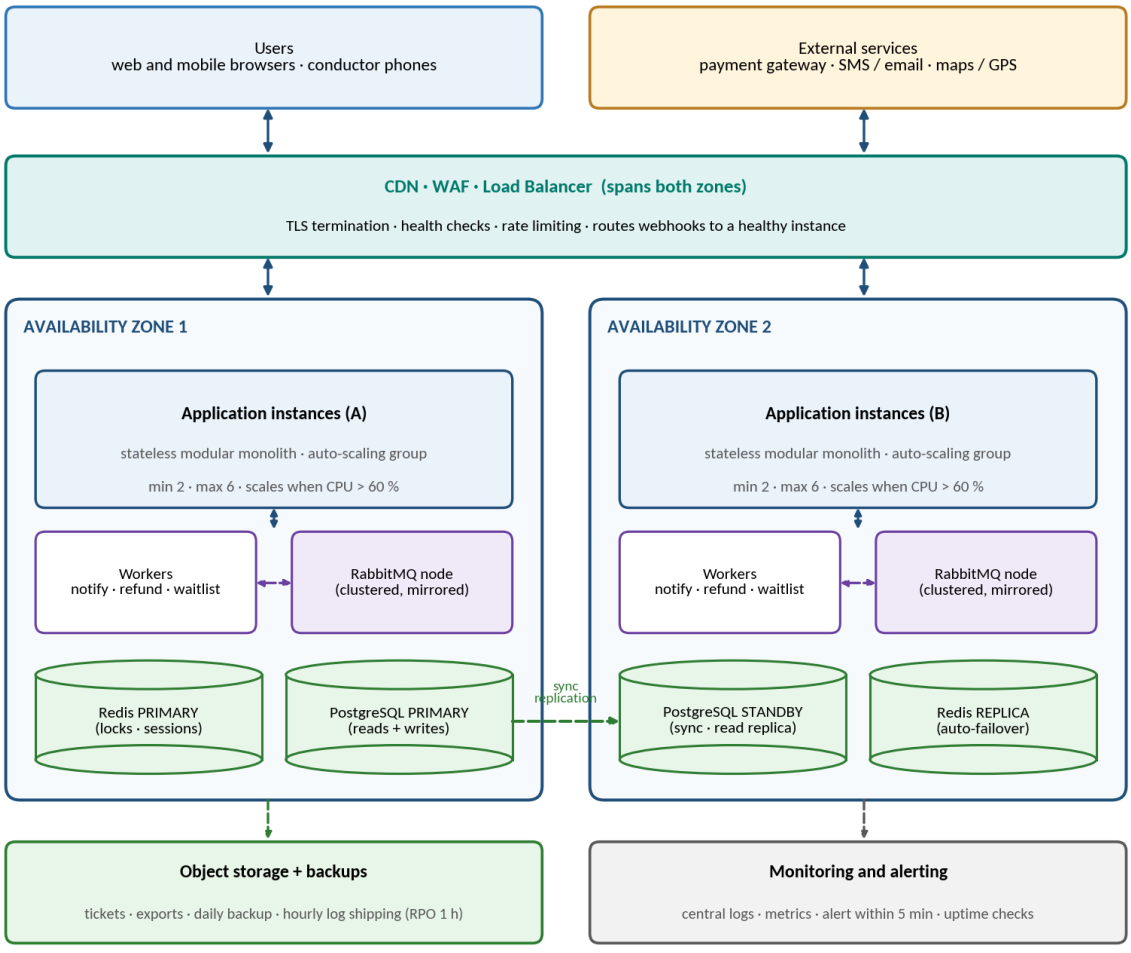


Figure 6: *Deployment view: redundant instances, databases and shared services across two availability zones*

5.7 Quality attribute scorecard

The scorecard connects each target to the mechanism that delivers it and to the test that will prove it before release.

Attribute	Key target	Main mechanism	How it will be verified
Scalability	5,000 concurrent users; 500 buses	Stateless instances, cache, auto-scaling	Load test with 5,000 virtual users
Security	0 high-severity findings; 0 card numbers stored	Layered defence, gateway-hosted payment	OWASP ZAP scan, penetration test, code review
Performance	Search 3 s (p95); seat map 2 s; confirm 5 s	Cache, WebSocket push, asynchronous workers	Load test with percentile reports and APM
Reliability	0 duplicate seat sales	Double-Gate Seat Guard, outbox, reconciliation	Concurrency test: 100 parallel requests for one seat give exactly 1 success
Maintainability	80% coverage; config changes without redeploy	Modules, layer rules, configuration store	Coverage report in the pipeline; configuration change drill
Availability	99.5% monthly uptime	Two zones, failover, graceful degradation	Failover drill; uptime monitor

6. Justify Architectural Decisions

6.1 Rationale behind the main decisions

Each important choice is recorded as an architecture decision, with the alternatives that were rejected and the reason.

ID	Decision	Alternatives considered	Why this was chosen
AD-1	Hybrid: layered modular monolith with event-driven workers	Microservices; serverless; plain monolith	Best fit for consistency, 12-week delivery and cost (score 4.30); events fix the weak spots of a plain monolith
AD-2	Double-Gate Seat Guard: Redis lock plus database UNIQUE rule	Database locks only; Redis only	Database-only locks hold connections and are slow under contention; Redis-only can lose a lock on failure. Two gates give speed and safety
AD-3	PostgreSQL as system of record	NoSQL database	Relational data, ACID transactions and UNIQUE constraints are essential; open source (T-2)
AD-4	Redis for locks, sessions and cache	Cache inside each instance	Must be shared by all instances; native key expiry fits the 8-minute lock (T-4)
AD-5	Message queue with transactional outbox	Direct calls to SMS and gateway; Kafka	Failures of vendors never block bookings; no lost events; RabbitMQ is simpler than Kafka for this volume
AD-6	Webhook-driven payment confirmation plus reconciliation	Trusting the browser redirect	A redirect can be faked or lost; only a verified webhook may confirm a booking (FR-23)
AD-7	WebSocket push for the seat map, with polling as fallback	Polling only	Meets the 2-second refresh with less load on the servers
AD-8	Progressive web apps for all three clients	Native Android and iOS apps	One code base, wider reach, lower cost; browser camera is enough for QR scanning
AD-9	Business rules held in configuration	Rules hard-coded	Lock time, slabs and fees change without redeployment (NFR-19)
AD-10	Stateless API with JWT and Redis sessions	In-memory server sessions	Any instance can serve any request, which makes horizontal scaling and failover simple

6.2 Trade-offs considered

No architecture is free. The table states what each major choice costs and how the cost is reduced.

Choice	What we gain	What we give up	How the downside is reduced
Modular monolith instead of microservices	Strong consistency, fast delivery, low cost	Independent scaling and deployment of each part; one serious bug can affect every instance	Strict module boundaries, automated tests, rolling deployment, at least 2 instances
Two gates for seat safety	Speed and a guarantee of no double booking	An extra component (Redis) to run and monitor	Managed Redis with a replica; database fallback if Redis is down
Event-driven workers	Resilience and loose coupling	Eventual consistency (an SMS may arrive seconds later) and more moving parts	Outbox, idempotent handlers, dead-letter queue, queue-length alerts
WebSocket push	Near real-time seat map	Persistent connections use more memory	Fallback to short polling; scale connection handling with the instances
Progressive web apps	One code base, low cost	Less access to native device features	Camera scanning works in the browser; tested on the NFR-21 device list
Third-party gateway	Card data stays out of scope (PCI-DSS)	Dependency and per-transaction fees (E-2)	Adapter interface allows a second gateway; retries and reconciliation

6.3 Support for future enhancements, integration and long-term maintainability

The architecture is built to change. API-first design, adapters, events and configuration keep most future changes small.

Future need	How the architecture supports it	Change needed
Counter or agent booking (assumption A-01)	Same API and role model; a new client uses the same booking flow	Add a role and a screen
New payment method or second gateway	Payment Adapter interface isolates the vendor	Write one new adapter
Regional-language support (suggested in Assessment Tool 1)	Text and message templates are separate from logic	Add language packs to the clients and templates
Seasonal or dynamic pricing	Fares and rules live in the Fleet module and configuration; events carry price changes	Add a pricing rule component
Partner and aggregator integration	Public REST API versioned as /v1; the gateway handles keys and rate limits	Publish and document the API
Demand forecasting and analytics	Events and the read replica provide data without touching the primary database	Add an event consumer or data warehouse feed
Several operators on one platform	Modules already separate data by domain concept	Add an operator identifier to tables and roles

When to split into services. The monolith is a starting point, not a dead end. Because modules use clear interfaces and events, each can be moved out when a measurable trigger is reached.

Module	Extract when	Why it separates cleanly
Notification	Queue backlog stays above 5 minutes, or message volume grows sharply	Already a separate worker that only consumes events
Search & Catalogue	Search load uses most of the cluster's CPU	Read-only and cache-friendly; can have its own replica
Payment	A second gateway is added or the payment audit scope must be narrowed	Already behind an adapter and a webhook endpoint
Tracking & Alerts	GPS data arrives at high frequency from many buses	Different data shape (time series) and no shared transactions
Seat Inventory and Booking	Only if one primary database can no longer take the write load	Keep together as long as possible because they need one transaction

6.4 Main risks and mitigation

Risk	Impact	Mitigation
Redis fails while seats are locked	Locks lost; contention moves to the database	Replica with automatic failover; database fallback lock; UNIQUE rule still prevents duplicates
Payment webhook is lost or delayed	Customer pays but has no ticket	Reconciliation worker polls the gateway; automatic refund within 15 minutes
Festival traffic spike	Slow search and seat map	Auto-scaling, caching, CDN; load test before launch with 5,000 users
Payment or SMS vendor outage	Bookings or alerts delayed	Timeouts, circuit breakers, retries, email fallback, second gateway option
Data breach	Loss of personal data and trust	Layered security, encryption, audit log, minimal data kept, breach response plan

6.5 Conclusion

BusBuddy needs an architecture that treats seat consistency as the top priority while staying simple enough to build in six sprints and flexible enough to grow. The **Hybrid architecture** (a layered modular monolith with event-driven workers) meets this need better than the alternatives, scoring 4.30 out of 5 in the weighted evaluation. The **Double-Gate Seat Guard** removes the double-booking problem at its source, the event-driven edge keeps third-party failures away from the customer, and the stateless design in two availability zones supports the targets of 5,000 users and 99.5% uptime. Clear module boundaries, configuration-driven rules and recorded decisions keep the system maintainable, and the extraction triggers in Section 6.3 give a safe path to microservices if BusBuddy outgrows a single application.

References

- Bass, L., Clements, P. and Kazman, R. (2021). Software Architecture in Practice, 4th ed. Addison-Wesley.
- Sommerville, I. (2015). Software Engineering, 10th ed. Pearson.
- Pressman, R. S. and Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach, 9th ed. McGraw-Hill.
- Richardson, C. (2018). Microservices Patterns (transactional outbox, saga). Manning.
- ISO/IEC/IEEE 42010:2022, Architecture description; ISO/IEC 25010:2011, Systems and software quality models.
- PCI Security Standards Council, PCI Data Security Standard; OWASP Top 10; Digital Personal Data Protection Act, 2023 (India).